



PyTorch 卷积网络配置与训练 (一)

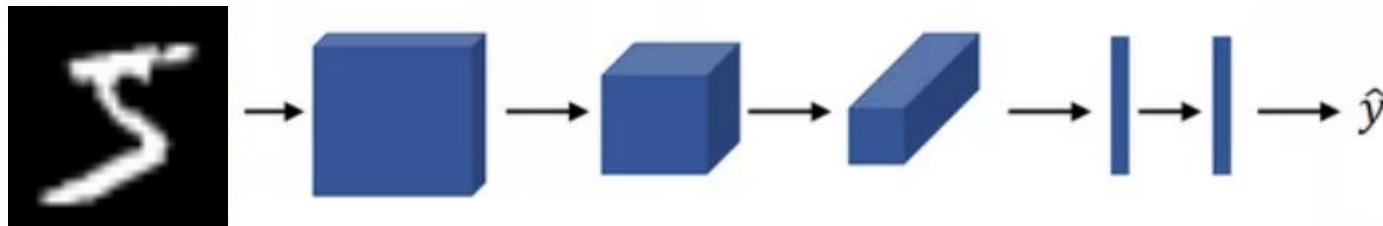


人工智能与Python程序设计 教研组

卷积神经网络训练

- 训练集: x 表示一张图片, y 是类别标注
- 损失函数: 如果使用交叉熵损失函数, 不需要做softmax
- 优化器: 例如梯度下降法或Adam

Training set $(x^{(1)}, y^{(1)}) \dots (x^{(m)}, y^{(m)})$.



$$\text{Cost } J = \frac{1}{m} \sum_{i=1}^m \mathcal{L}(\hat{y}^{(i)}, y^{(i)})$$



提纲



人工智能与
Python程序设计

- 1. 卷积网络的训练流程
- 2. CIFAR-10数据介绍与加载
- 3. 网络搭建与模型优化
- 4. 图像识别大作业



提纲



人工智能与
Python程序设计

- 1. 卷积网络的训练流程
- 2. CIFAR-10数据介绍与加载
- 3. 网络搭建与模型优化
- 4. 图像识别大作业

卷积网络的训练流程



卷积网络的训练流程



Network

卷积网络的训练流程

网络前馈预测



Network



Dog!

卷积网络的训练流程

网络前馈预测



Network



Cat!



误差反向传播

卷积网络的训练流程



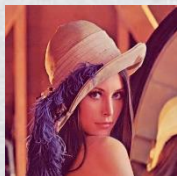
卷积网络的训练流程

数据准备

网络定义

损失函数

优化方法



It was the best of times, it was the worst of times, it was the age of wisdom, it was the age of foolishness...



Classification

Log Loss

Focal Loss

KL Divergence/
Relative
Entropy

Exponential
Loss

Hinge Loss

Regression

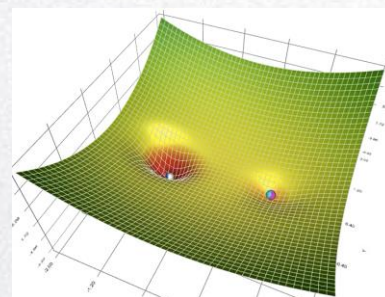
Mean Square
Error/
Quadratic Loss

Mean Absolute
Error

Huber Loss/
Smooth Mean
Absolute Error

Log cosh Loss

Quantile Loss





提纲

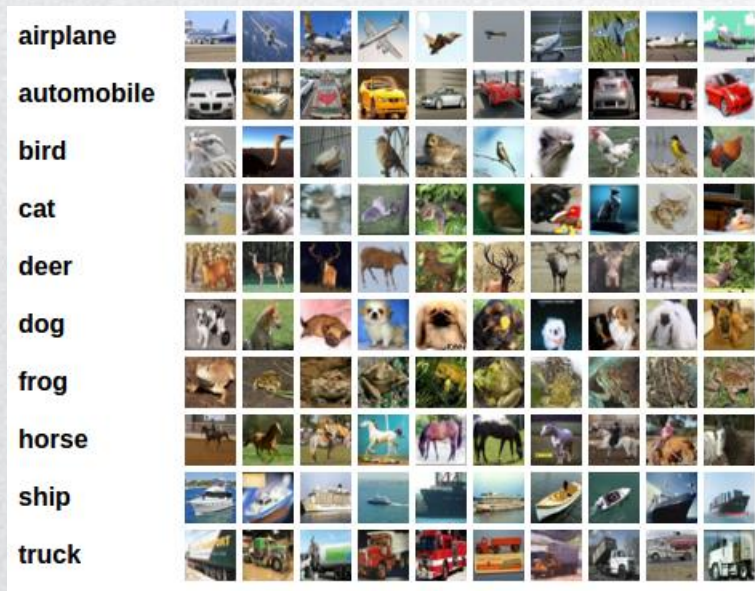


人工智能与
Python程序设计

- 1. 卷积网络的训练流程
- 2. CIFAR-10数据介绍与加载
- 3. 网络搭建与模型优化
- 4. 图像识别大作业

CIFAR-10数据介绍与加载

- CIFAR-10数据介绍
 - 10个类别，每个类别6000张图像，共计60000张。
 - 图像分辨率为32x32, 均为RGB三通道图像。





CIFAR-10数据介绍与加载

- torchvision库

— 集成计算机视觉相关的数据集，模型架构，以及常用的图像处理操作等

- torchvision.datasets

- CelebA
- CIFAR
- Cityscapes
- COCO
- DatasetFolder
- EMNIST
- FakeData
- Fashion-MNIST
- Flickr
- HMDB51
- ImageFolder
- ImageNet
- Kinetics-400
- KMNIST

- torchvision.io

- Video
- Fine-grained video API
- Image

- torchvision.models

- Classification
- Semantic Segmentation
- Object Detection, Instance Segmentation and Person Keypoint Detection
- Video classification

- torchvision.transforms

- Scriptable transforms
- Compositions of transforms
- Transforms on PIL Image and torch.*Tensor
- Transforms on PIL Image only
- Transforms on torch.*Tensor only
- Conversion Transforms
- Generic Transforms
- Functional Transforms



CIFAR-10数据介绍与加载

- 基于torchvision库对CIFAR-10数据加载

root=是保存文件夹路径，train=true 是训练集（已经划分出来了数据集和训练集）

download=true 就是开始下载

```
import torch
import torchvision
```

```
trainset = torchvision.datasets.CIFAR10(root='./data', train=True,
                                         download=True)
trainloader = torch.utils.data.DataLoader(trainset, batch_size=4,
                                           shuffle=True, num_workers=2)

testset = torchvision.datasets.CIFAR10(root='./data', train=False,
                                         download=True)
testloader = torch.utils.data.DataLoader(testset, batch_size=4,
                                         shuffle=False, num_workers=2)
```

数据集类
实例化



CIFAR-10数据介绍与加载

- 构建CIFAR-10数据集

以后数据集必须继承 dataset

```
class CIFAR10(VisionDataset):  
    ..  
    ..  
    ..  
    def __getitem__(self, index: int) -> Tuple[Any, Any]:  
        """  
        Args:  
            index (int): Index  
  
        Returns:  
            tuple: (image, target) where target is index of the target class.  
        """  
        img, target = self.data[index], self.targets[index]  
  
        # doing this so that it is consistent with all other datasets  
        # to return a PIL Image  
        img = Image.fromarray(img)  
  
        if self.transform is not None:  
            img = self.transform(img)  
  
        if self.target_transform is not None:  
            target = self.target_transform(target)  
  
        return img, target  
  
    def __len__(self) -> int:  
        return len(self.data)
```

根据index获取对应数据样本，并返回到数据队列

返回图片和 label

必须写
数据个数声明



CIFAR-10数据介绍与加载

- 基于torchvision库对CIFAR-10数据加载

```
import torch
import torchvision

trainset = torchvision.datasets.CIFAR10(root='./data', train=True,
                                         download=True)
trainloader = torch.utils.data.DataLoader(trainset, batch_size=4,
                                           shuffle=True, num_workers=2)

testset = torchvision.datasets.CIFAR10(root='./data', train=False,
                                         download=True)
testloader = torch.utils.data.DataLoader(testset, batch_size=4,
                                         shuffle=False, num_workers=2)
```

DataLoader
实例化

迭代器，包括
batchsize 的数
据，shuffle=true
是每次取出来的
batchsize 里面
的元素 随机，num
表示同时缓存几
个 batch



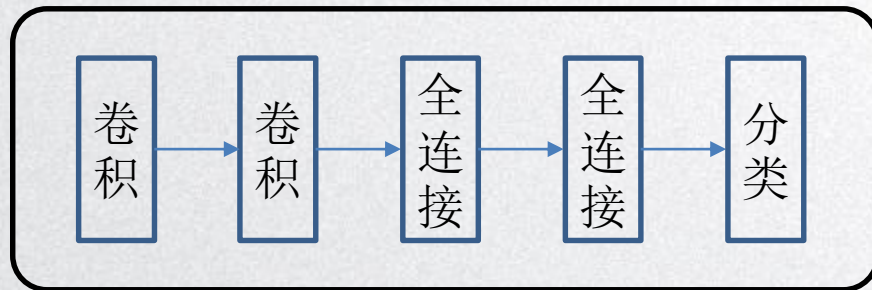
提纲



人工智能与
Python程序设计

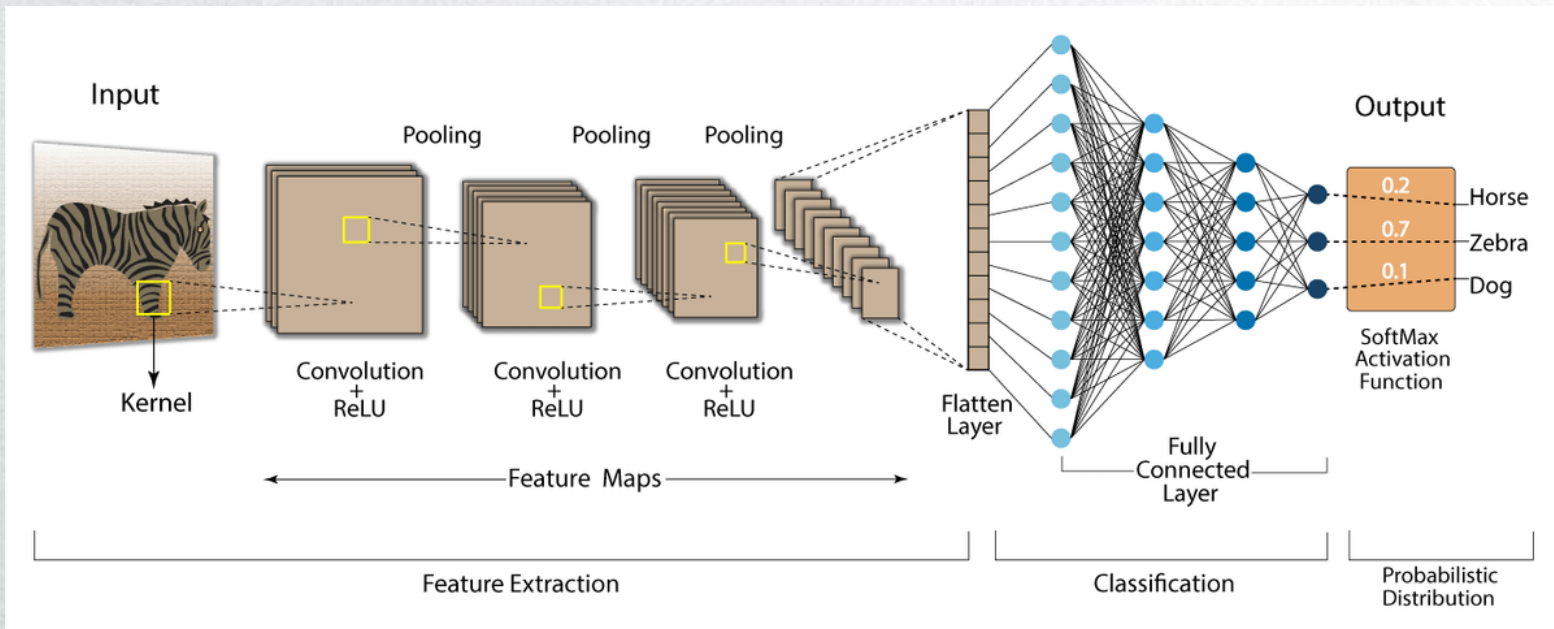
- 1. 卷积网络的训练流程
- 2. CIFAR-10数据介绍与加载
- 3. 网络搭建与模型优化
- 4. 图像识别大作业

网络搭建与模型优化

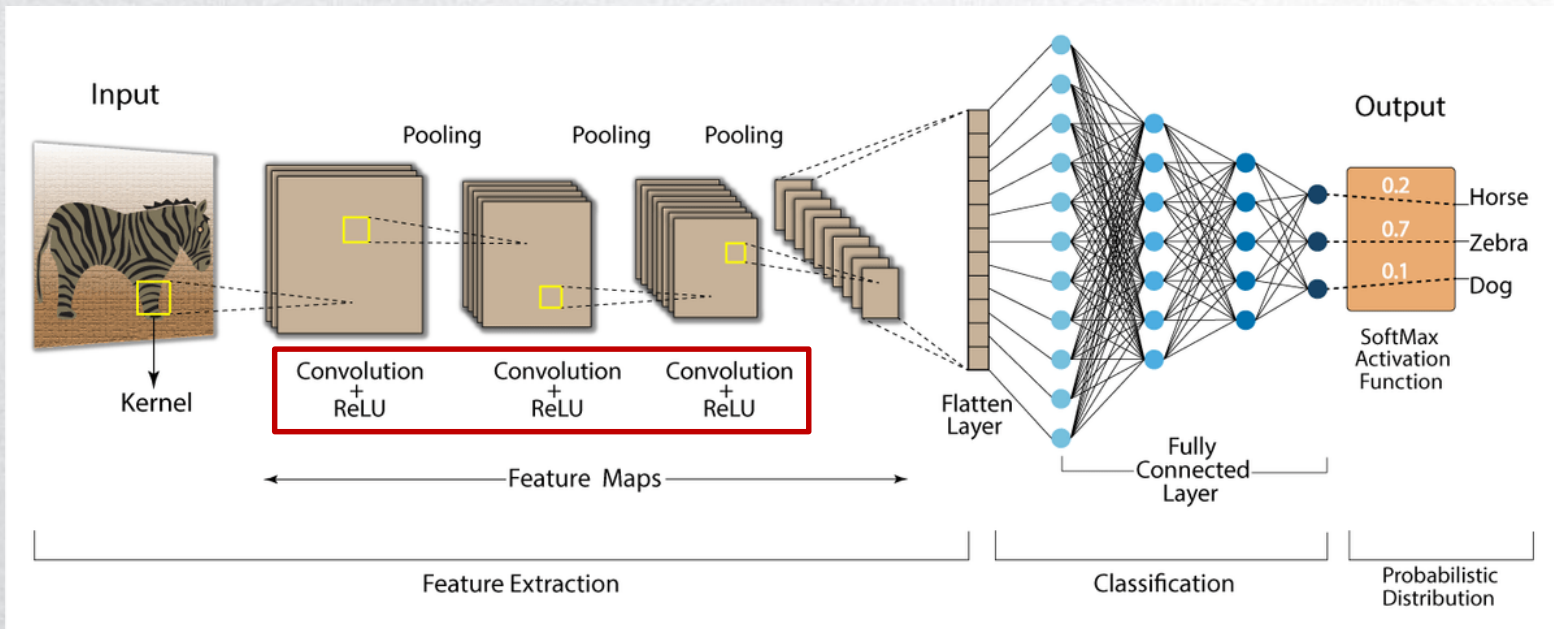


类别
预测

网络搭建与模型优化

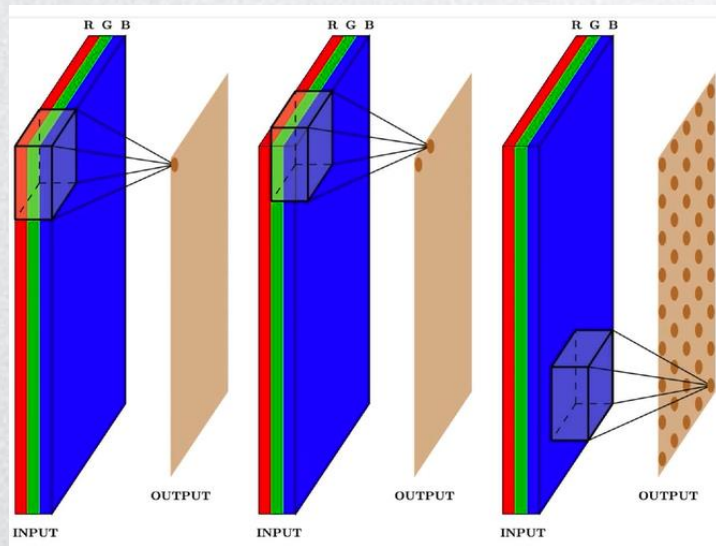


网络搭建与模型优化



网络搭建与模型优化

- 卷积

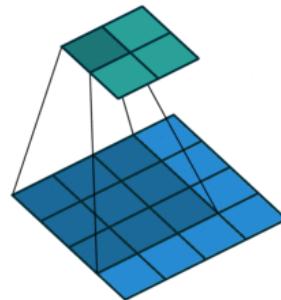


网络搭建与模型优化

• 卷积

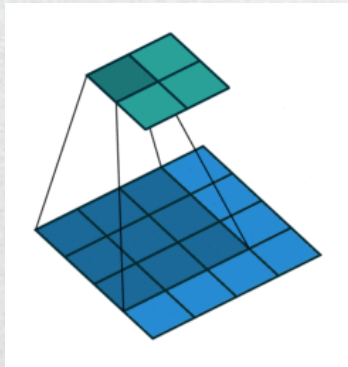
```
CLASS torch.nn.Conv2d(in_channels: int, out_channels: int, kernel_size: Union[T, Tuple[T, T]], stride: Union[T, Tuple[T, T]] = 1, padding: Union[T, Tuple[T, T]] = 0, dilation: Union[T, Tuple[T, T]] = 1, groups: int = 1, bias: bool = True, padding_mode: str = 'zeros') = [SOURCE]
```

- **in_channels** (*int*) – Number of channels in the input image
- **out_channels** (*int*) – Number of channels produced by the convolution
- **kernel_size** (*int or tuple*) – Size of the convolving kernel
- **stride** (*int or tuple, optional*) – Stride of the convolution. Default: 1
- **padding** (*int or tuple, optional*) – Zero-padding added to both sides of the input. Default: 0
- **padding_mode** (*string, optional*) – 'zeros', 'reflect', 'replicate' or 'circular'. Default: 'zeros'
- **dilation** (*int or tuple, optional*) – Spacing between kernel elements. Default: 1
- **groups** (*int, optional*) – Number of blocked connections from input channels to output channels. Default: 1
- **bias** (*bool, optional*) – If `True`, adds a learnable bias to the output. Default: `True`

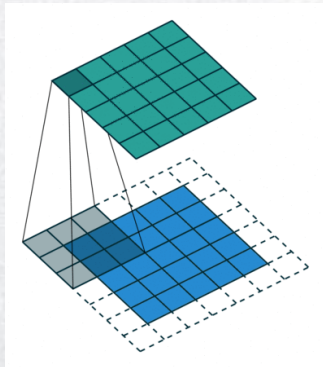


网络搭建与模型优化

- 卷积

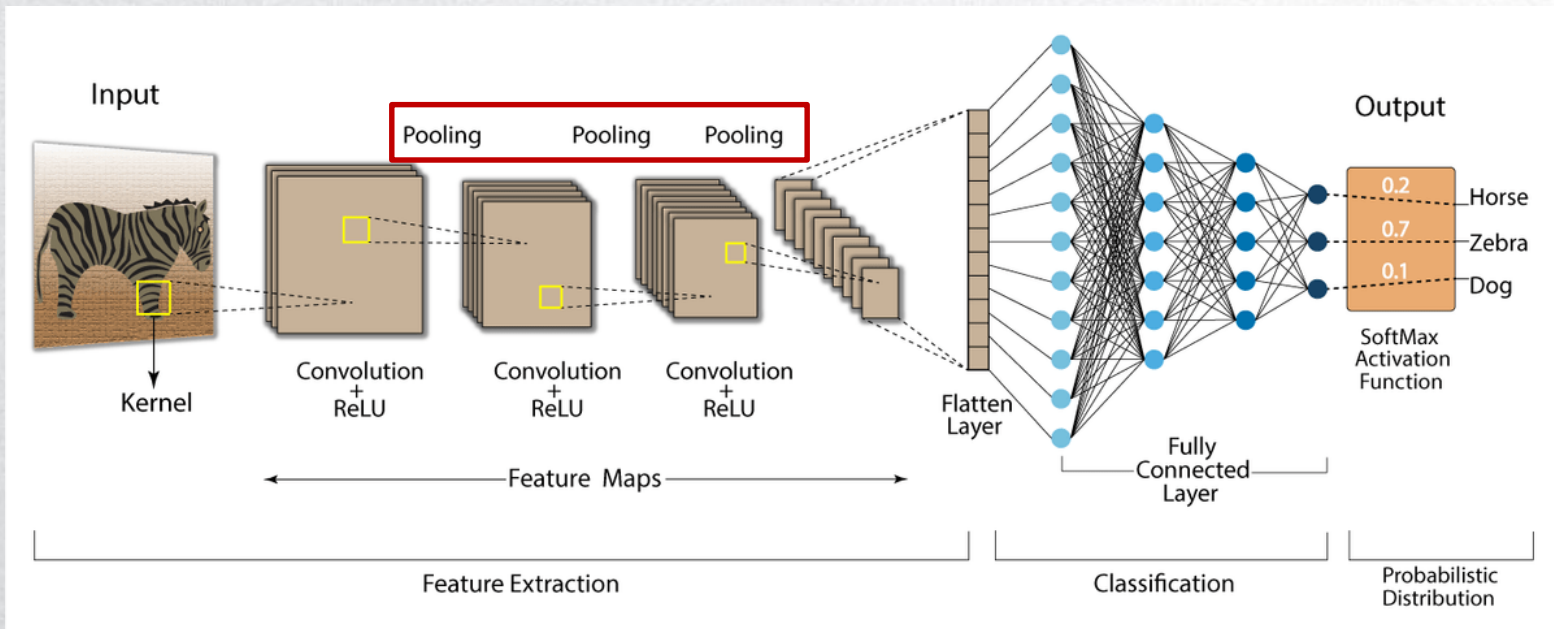


No padding,
stride 1



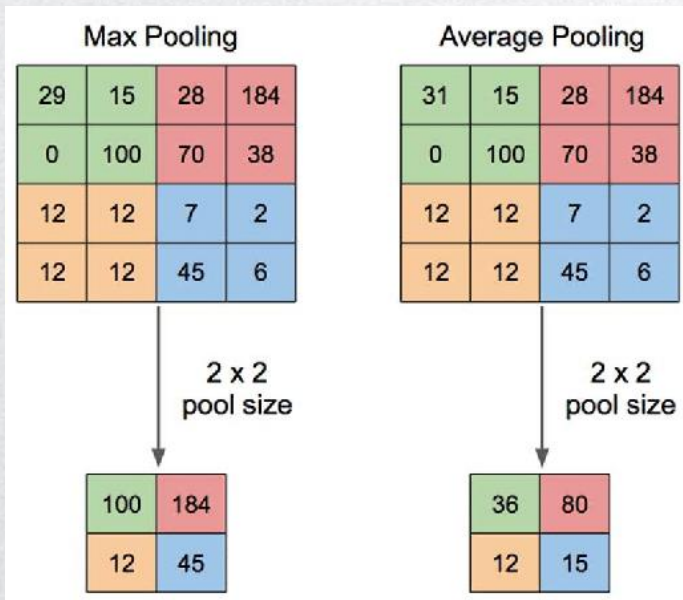
padding: 1
stride: 1

网络搭建与模型优化



网络搭建与模型优化

- 池化(Pooling)





网络搭建与模型优化

- 池化(Pooling)

最大池化

```
CLASS torch.nn.MaxPool2d(kernel_size, stride=None, padding=0, dilation=1,  
                           return_indices=False, ceil_mode=False)
```

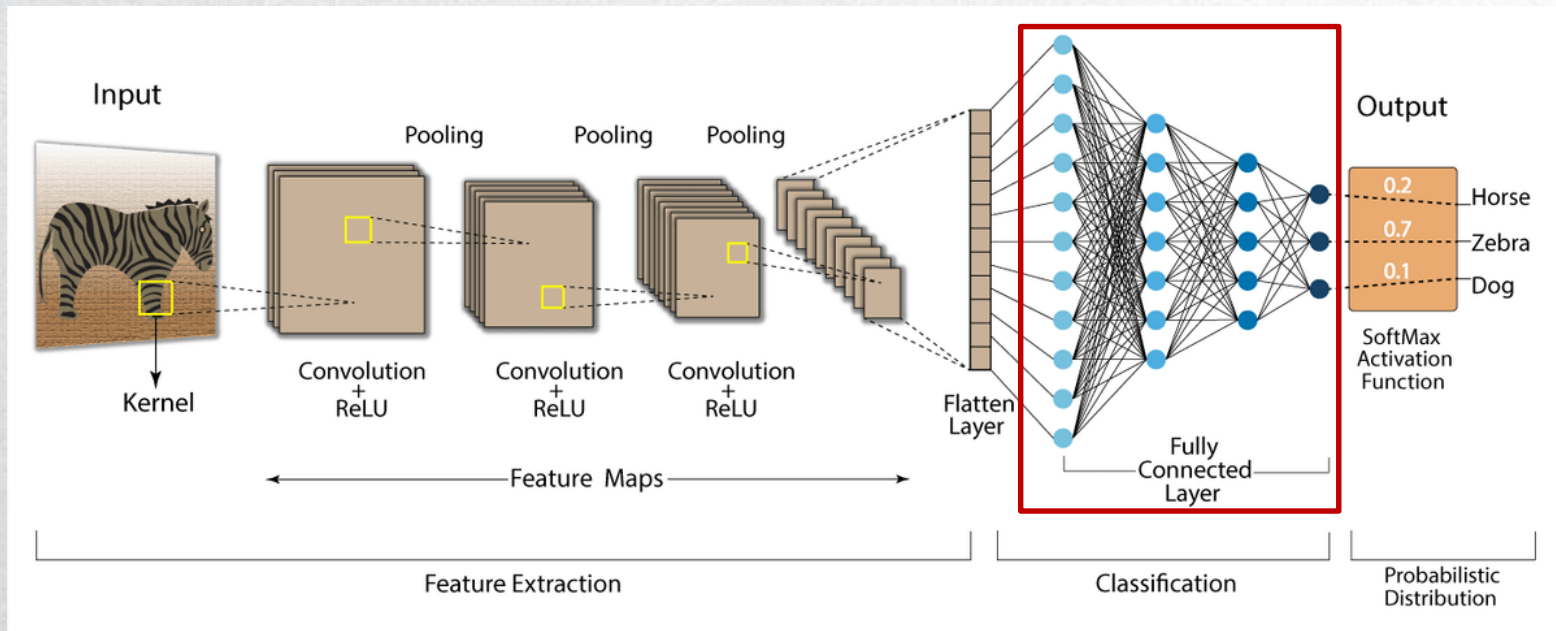
- **kernel_size** – the size of the window to take a max over
- **stride** – the stride of the window. Default value is `kernel_size`
- **padding** – implicit zero padding to be added on both sides
- **dilation** – a parameter that controls the stride of elements in the window
- **return_indices** – if `True`, will return the max indices along with the outputs. Useful for `torch.nn.MaxUnpool2d` later
- **ceil_mode** – when `True`, will use `ceil` instead of `floor` to compute the output shape

平均池化

```
CLASS torch.nn.AvgPool2d(kernel_size, stride=None, padding=0, ceil_mode=False,  
                           count_include_pad=True, divisor_override=None)
```

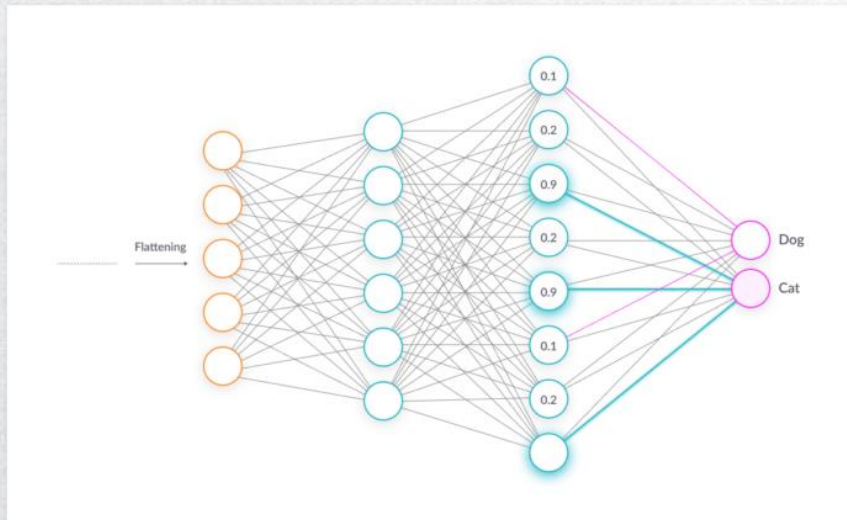
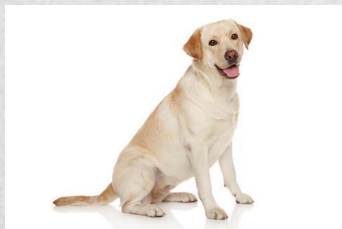
Applies a 2D average pooling over an input signal composed of several input planes.

网络搭建与模型优化



网络搭建与模型优化

- 全连接(Linear, Fully Connected Layer)





网络搭建与模型优化

- 全连接(Linear, Fully Connected Layer)

```
CLASS torch.nn.Linear(in_features: int, out_features: int, bias: bool = True)
```

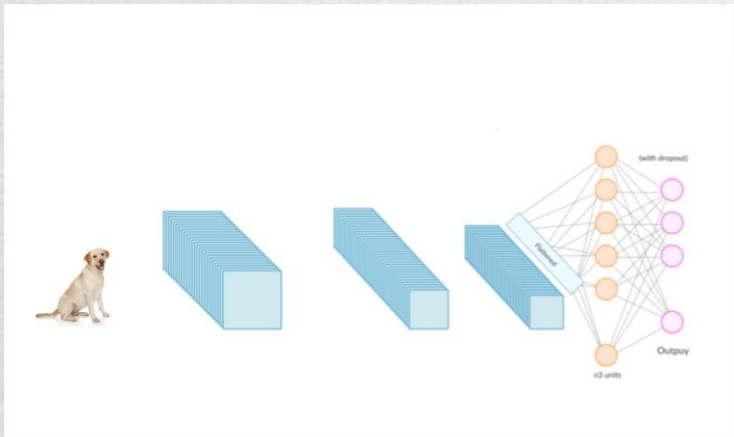
[SOURCE]

Applies a linear transformation to the incoming data: $y = xA^T + b$

- **in_features** – size of each input sample
- **out_features** – size of each output sample
- **bias** – If set to `False`, the layer will not learn an additive bias. Default: `True`

bias 是 out features
维数的

网络搭建与模型优化



```
import torch.nn as nn
import torch.nn.functional as F
```

```
class Net(nn.Module):
    def __init__(self):
        super(Net, self).__init__()
        self.conv1 = nn.Conv2d(3, 6, 5)
        self.pool = nn.MaxPool2d(2, 2)
        self.conv2 = nn.Conv2d(6, 16, 5)
        self.fc1 = nn.Linear(16 * 5 * 5, 120)
        self.fc2 = nn.Linear(120, 84)
        self.fc3 = nn.Linear(84, 10)

    def forward(self, x):
        x = self.pool(F.relu(self.conv1(x)))
        x = self.pool(F.relu(self.conv2(x)))
        x = x.view(-1, 16 * 5 * 5)
        x = F.relu(self.fc1(x))
        x = F.relu(self.fc2(x))
        x = self.fc3(x)
        return x
```

```
net = Net()
```



网络搭建与模型优化

在构造函数中，
实例化不同的
layer组件，并赋
给类成员变量

```
import torch.nn as nn
import torch.nn.functional as F
```

```
class Net(nn.Module):
    def __init__(self):
        super(Net, self).__init__()
        self.conv1 = nn.Conv2d(3, 6, 5)
        self.pool = nn.MaxPool2d(2, 2)
        self.conv2 = nn.Conv2d(6, 16, 5)
        self.fc1 = nn.Linear(16 * 5 * 5, 120)
        self.fc2 = nn.Linear(120, 84)
        self.fc3 = nn.Linear(84, 10)
```

```
def forward(self, x):
    x = self.pool(F.relu(self.conv1(x)))
    x = self.pool(F.relu(self.conv2(x)))
    x = x.view(-1, 16 * 5 * 5)
    x = F.relu(self.fc1(x))
    x = F.relu(self.fc2(x))
    x = self.fc3(x)
    return x
```

```
net = Net()
```

在前馈函数中，利用实例化的
组件对网络进行搭建，并对输入
Tensor进行操作，并返回
Tensor类型的输出结果

-1 也可以写成 shape (0) ，这个是batchsize



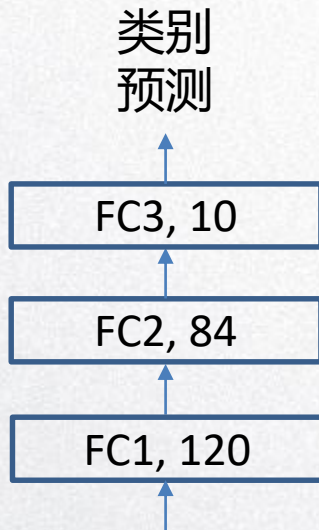
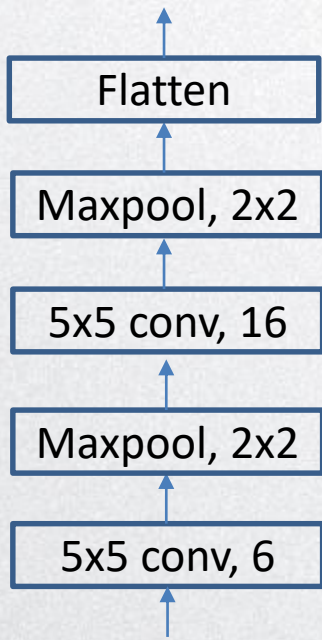
网络搭建与模型优化

```
import torch.nn as nn
import torch.nn.functional as F
```

```
class Net(nn.Module):
    def __init__(self):
        super(Net, self).__init__()
        self.conv1 = nn.Conv2d(3, 6, 5)
        self.pool = nn.MaxPool2d(2, 2)
        self.conv2 = nn.Conv2d(6, 16, 5)
        self.fc1 = nn.Linear(16 * 5 * 5, 120)
        self.fc2 = nn.Linear(120, 84)
        self.fc3 = nn.Linear(84, 10)
```

```
    def forward(self, x):
        x = self.pool(F.relu(self.conv1(x)))
        x = self.pool(F.relu(self.conv2(x)))
        x = x.view(-1, 16 * 5 * 5)
        x = F.relu(self.fc1(x))
        x = F.relu(self.fc2(x))
        x = self.fc3(x)
        return x
```

```
net = Net()
```



网络搭建与模型优化

- 损失函数

- 根据网络模型对当前数据样本下的预测输出结果 p , 计算其与真实目标 y 之间的距离。该距离表示当前网络输出结果的质量
- 在基于图像数据的多类别分类任务中, 通常采用交叉熵损失函数对此距离进行计算
- 交叉熵损失: 衡量的是输出为0到1之间概率值的分类模型的性能。随着预测概率与实际标签的偏离, 交叉熵损失会增加。
 - 对于二分类

$$-(y \log(p) + (1 - y) \log(1 - p))$$

- 对于多分类:

$$-\sum_{c=1}^M y_c \log(p_c)$$



网络搭建与模型优化

- 损失函数

- 交叉熵损失：衡量的是输出为0到1之间概率值的分类模型的性能。随着预测概率与实际标签的偏离，交叉熵损失会增加。

- 对于二分类:

$$-(y \log(p) + (1 - y) \log(1 - p))$$

- 对于多分类:

$$-\sum_{c=1}^M y_c \log(p_c)$$



网络搭建与模型优化

- 损失函数
 - PyTorch中的交叉熵函数

```
CLASS torch.nn.CrossEntropyLoss(weight: Optional[torch.Tensor] = None, size_average=None,  
ignore_index: int = -100, reduce=None, reduction: str = 'mean') [SOURCE]
```

The loss can be described as:

$$\text{loss}(x, \text{class}) = -\log \left(\frac{\exp(x[\text{class}])}{\sum_j \exp(x[j])} \right) = -x[\text{class}] + \log \left(\sum_j \exp(x[j]) \right)$$

softmax
function



网络搭建与模型优化

- 损失函数
 - PyTorch中的交叉熵函数
 - 交叉熵损失函数的实例化

```
import nn  
  
criterion = nn.CrossEntropyLoss()
```

- 优化方法的声明与实例化

```
import torch.optim as optim  
  
optimizer = optim.SGD(net.parameters(), lr=0.001, momentum=0.9)
```


网络搭建与模型优化

- 模型训练

网络定义

损失函数

优化方法

一个 batchsize

单批量数据下的一次模型训练

数据输入

模型处理

损失计算

梯度计算

参数优化

epoch 遍历数据集多少次

完整数据下的迭代模型训练



网络搭建与模型优化

- 模型训练

```
for epoch in range(2): # loop over the dataset multiple times

    running_loss = 0.0
    for i, data in enumerate(trainloader, 0):
        # get the inputs; data is a list of [inputs, labels]
        inputs, labels = data

        # zero the parameter gradients
        optimizer.zero_grad()

        # forward + backward + optimize
        outputs = net(inputs)
        loss = criterion(outputs, labels)
        loss.backward()
        optimizer.step()

        # print statistics
        running_loss += loss.item()
        if i % 2000 == 1999: # print every 2000 mini-batches
            print('[%d, %5d] loss: %.3f' %
                  (epoch + 1, i + 1, running_loss / 2000))
            running_loss = 0.0

    print('Finished Training')
```

网络搭建与模型优化

- 模型训练

全部数据
迭代次数

```
for epoch in range(2): # loop over the dataset multiple times
```

```
    running_loss = 0.0
```

```
    for i, data in enumerate(trainloader, 0):
```

```
        # get the inputs; data is a list of [inputs, labels]
        inputs, labels = data
```

获取当前批次数据

```
        # zero the parameter gradients
        optimizer.zero_grad()
```

清空模型参数的梯度

```
        # forward + backward + optimize
        outputs = net(inputs)
        loss = criterion(outputs, labels)
        loss.backward()
        optimizer.step()
```

模型预测与参数优化

```
    # print statistics
```

```
    running_loss += loss.item()
```

```
    if i % 2000 == 1999: # print every 2000 mini-batches
```

```
        print('[%d, %5d] loss: %.3f' %
              (epoch + 1, i + 1, running_loss / 2000))
        running_loss = 0.0
```

```
print('Finished Training')
```



网络搭建与模型优化

- 简单卷积网络训练流程回顾
 - 参见simpleConvNet.py



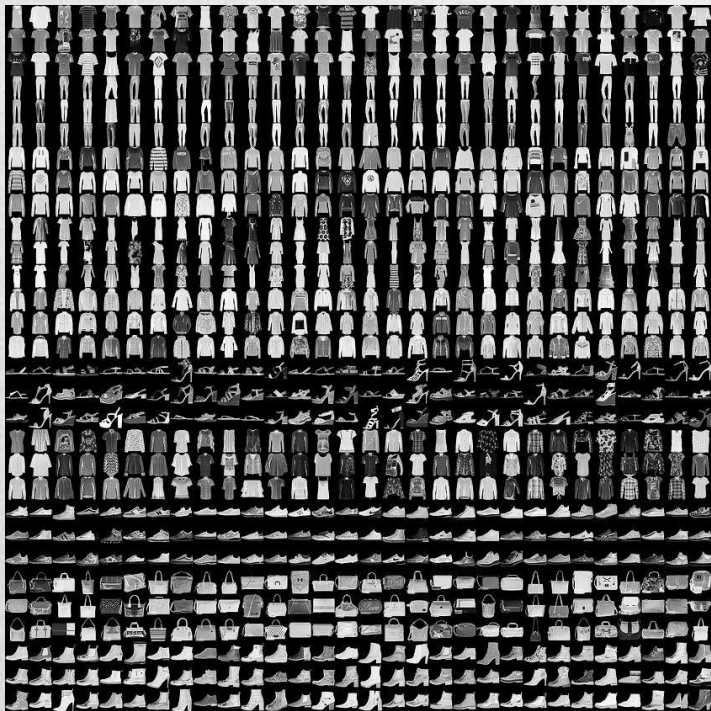
提纲



人工智能与
Python程序设计

- 1. 卷积网络的训练流程
- 2. 简单卷积网络的搭建
- 3. 损失定义与模型优化
- 4. 图像识别大作业

商品图像 (Fashion-Mnist) 分类



FashionMNIST 是一个图像数据集。它是由 Zalando (一家德国的时尚科技公司) 旗下的研究部门提供, 涵盖了来自 10 种类别的共 7 万个不同商品的正面图片。

商品图像 (Fashion-Mnist) 分类



- 实现一个基于CNN的图像分类模型
- 具体要求：
 - 模型构建：继承`nn.Module`，实现`FashionMnistModel`类
 - 模型训练：基于Fashion-Mnist数据集，完成`FashionMnistModel`的训练
 - 模型测试：实现`test`函数，完成图像分类测试



机器学习的基本流程

数据准备

- 数据标注
- 训练集/验证集/测试集分割
- 特征提取



模型训练

- 分类损失函数
- 损失函数优化和参数调优



模型测试

- 性能评价指标
- 交叉验证

数据准备

- 构建数据集类:

构造函数:

实例化数据加载的相关类属性

```
class FashionDataset(Dataset):
    """
    定义Dataset:
    - 用于加载训练和测试数据, 请勿改动
    - 返回一张图片(3维Tensor)以及对应的标签(0-9)
    """

    def __init__(self, datadir, transform, is_train = True):
        super().__init__()
        self.datadir = datadir
        self.img, self.label = self.load_data(self.datadir, is_train = is_train)
        self.len_data = len(self.img)
        self.transform = transform

    def __getitem__(self, index):
        return self.transform(self.img[index]), self.label[index]

    def __len__(self):
        return self.len_data

    def load_data(self, datadir, is_train):
        dirname = os.path.join(datadir)
        files = ['train-labels-idx1-ubyte.gz', 'train-images-idx3-ubyte.gz',
                't10k-labels-idx1-ubyte.gz', 't10k-images-idx3-ubyte.gz']

        paths = []
        for fname in files:
            paths.append(os.path.join(dirname, fname))
            if is_train:

                with gzip.open(paths[0], 'rb') as lbpath:
                    label = np.frombuffer(lbpath.read(), np.uint8, offset=8)

                with gzip.open(paths[1], 'rb') as imgpath:
                    img = np.frombuffer(imgpath.read(), np.uint8,
                                       offset=16).reshape(len(label), 28, 28)

            else:

                with gzip.open(paths[2], 'rb') as lbpath:
                    label = np.frombuffer(lbpath.read(), np.uint8, offset=8)

                with gzip.open(paths[3], 'rb') as imgpath:
                    img = np.frombuffer(imgpath.read(), np.uint8,
                                       offset=16).reshape(len(label), 28, 28)

        return img, label
```

数据准备

- 构建数据集类:

构造函数:

实例化数据加载的相关类属性

```
class FashionDataset(Dataset):  
    '''  
    定义Dataset:  
    - 用于加载训练和测试数据, 请勿改动  
    - 返回一张图片(3维Tensor)以及对应的标签(0-9)  
    '''  
  
    def __init__(self, datadir, transform, is_train = True):  
        super().__init__()  
        self.datadir = datadir  
        self.img, self.label = self.load_data(self.datadir, is_train = is_train)  
        self.len_data = len(self.img)  
        self.transform = transform  
  
    def __getitem__(self, index):  
        return self.transform(self.img[index]), self.label[index]  
  
    def __len__(self):  
        return self.len_data  
  
    def load_data(self, datadir, is_train):  
        dirname = os.path.join(datadir)  
        files = ['train-labels-idx1-ubyte.gz', 'train-images-idx3-ubyte.gz',  
                't10k-labels-idx1-ubyte.gz', 't10k-images-idx3-ubyte.gz']  
  
        paths = []  
        for fname in files:  
            paths.append(os.path.join(dirname, fname))  
        if is_train:  
  
            with gzip.open(paths[0], 'rb') as lbpath:  
                label = np.frombuffer(lbpath.read(), np.uint8, offset=8)  
            with gzip.open(paths[1], 'rb') as imgpath:  
                img = np.frombuffer(imgpath.read(), np.uint8,  
                                   offset=16).reshape(len(label), 28, 28)  
        else:  
            with gzip.open(paths[2], 'rb') as lbpath:  
                label = np.frombuffer(lbpath.read(), np.uint8, offset=8)  
  
            with gzip.open(paths[3], 'rb') as imgpath:  
                img = np.frombuffer(imgpath.read(), np.uint8,  
                                   offset=16).reshape(len(label), 28, 28)  
        return img, label
```

根据index获取对应数据
样本, 并返回到数据队列

数据准备

- 构建数据集类:

构造函数:

实例化数据加载的相关类属性

获取所有数据个数

```
class FashionDataset(Dataset):  
    '''  
    定义Dataset:  
    - 用于加载训练和测试数据, 请勿改动  
    - 返回一张图片(3维Tensor)以及对应的标签(0-9)  
    '''  
  
    def __init__(self, datadir, transform, is_train = True):  
        super().__init__()  
        self.datadir = datadir  
        self.img, self.label = self.load_data(self.datadir, is_train = is_train)  
        self.len_data = len(self.img)  
        self.transform = transform  
  
    def __getitem__(self, index):  
        return self.transform(self.img[index]), self.label[index]  
  
    def __len__(self):  
        return self.len_data  
  
    def load_data(self, datadir, is_train):  
        dirname = os.path.join(datadir)  
        files = ['train-labels-idx1-ubyte.gz', 'train-images-idx3-ubyte.gz',  
                't10k-labels-idx1-ubyte.gz', 't10k-images-idx3-ubyte.gz']  
  
        paths = []  
        for fname in files:  
            paths.append(os.path.join(dirname, fname))  
        if is_train:  
            with gzip.open(paths[0], 'rb') as lbpath:  
                label = np.frombuffer(lbpath.read(), np.uint8, offset=8)  
            with gzip.open(paths[1], 'rb') as imgpath:  
                img = np.frombuffer(imgpath.read(), np.uint8,  
                                   offset=16).reshape(len(label), 28, 28)  
        else:  
            with gzip.open(paths[2], 'rb') as lbpath:  
                label = np.frombuffer(lbpath.read(), np.uint8, offset=8)  
  
            with gzip.open(paths[3], 'rb') as imgpath:  
                img = np.frombuffer(imgpath.read(), np.uint8,  
                                   offset=16).reshape(len(label), 28, 28)  
        return img, label
```

根据index获取对应数据
样本, 并返回到数据队列

数据准备

- 构建数据集类:

构造函数:

实例化数据加载的相关类属性

获取所有数据个数

读取所有数据

```
class FashionDataset(Dataset):
    """
    定义Dataset:
    - 用于加载训练和测试数据, 请勿改动
    - 返回一张图片(3维Tensor)以及对应的标签(0-9)
    """

    def __init__(self, datadir, transform, is_train = True):
        super().__init__()
        self.datadir = datadir
        self.img, self.label = self.load_data(self.datadir, is_train = is_train)
        self.len_data = len(self.img)
        self.transform = transform

    def __getitem__(self, index):
        return self.transform(self.img[index]), self.label[index]

    def __len__(self):
        return self.len_data

    def load_data(self, datadir, is_train):
        dirname = os.path.join(datadir)
        files = ['train-labels-idx1-ubyte.gz', 'train-images-idx3-ubyte.gz',
                 't10k-labels-idx1-ubyte.gz', 't10k-images-idx3-ubyte.gz']

        paths = []
        for fname in files:
            paths.append(os.path.join(dirname, fname))
            if is_train:

                with gzip.open(paths[0], 'rb') as lbpath:
                    label = np.frombuffer(lbpath.read(), np.uint8, offset=8)

                with gzip.open(paths[1], 'rb') as imgpath:
                    img = np.frombuffer(imgpath.read(), np.uint8,
                                        offset=16).reshape(len(label), 28, 28)

            else:

                with gzip.open(paths[2], 'rb') as lbpath:
                    label = np.frombuffer(lbpath.read(), np.uint8, offset=8)

                with gzip.open(paths[3], 'rb') as imgpath:
                    img = np.frombuffer(imgpath.read(), np.uint8,
                                        offset=16).reshape(len(label), 28, 28)

        return img, label
```

根据index获取对应数据
样本, 并返回到数据队列

数据准备

- 实例化DataLoader:
 - Train_loader
 - Test_loader

```
# 定义data loader
train_dataset = FashionDataset('data',
                                transform=transforms.Compose([
                                    transforms.ToTensor(),
                                    transforms.Normalize((0.1307,), (0.3081,))
                                ])
)

train_loader = DataLoader(train_dataset, batch_size=320, shuffle=True, num_workers= 4)

test_dataset = FashionDataset('data',
                               transform=transforms.Compose([
                                   transforms.ToTensor(),
                                   transforms.Normalize((0.1307,), (0.3081,))
                               ]),
                               is_train = False
)
test_loader = DataLoader(test_dataset, batch_size=32, shuffle=False, num_workers= 1)
```

模型搭建与训练

- 模型训练与测试类:

```
class Model():
    def __init__(self):
        """
        创建模型和优化器, 设置模型超参数
        * 参数
            * learning_rate
            * epoches
            * model_save_path
            * device: cuda or cpu
        * 模型
            * 创建FashionMnistModel的实例, 命名为model
            * 定义optimizer
            * 定义loss function
        """
        self.lr = 0.01
        self.epoches = 20
        self.model_save_path = './model'
        # 指定训练的device, 优先使用GPU, GPU不可用时加载CPU
        self.device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
        self.model = FashionMnistModel().to(self.device)
        self.optimizer = torch.optim.Adam(self.model.parameters(), lr=self.lr)
        self.loss_function = nn.CrossEntropyLoss()
```



模型搭建与训练

- 模型训练与测试类：

```
class Model():
    def __init__(self):
        '''
        创建模型和优化器，设置模型超参数
        * 参数
            * learning_rate
            * epoches
            * model_save_path
            * device: cuda or cpu
        * 模型
            * 创建FashionMnistModel的实例，命名为model
            * 定义optimizer
            * 定义loss function
        '''
        self.lr = 0.01
        self.epoches = 20
        self.model_save_path = './model'
        # 指定训练的device, 优先使用GPU, GPU不可用时加载CPU
        self.device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
        self.model = FashionMnistModel().to(self.device)
        self.optimizer = torch.optim.Adam(self.model.parameters(), lr=self.lr)
        self.loss_function = nn.CrossEntropyLoss()
```

需同学自主完成 FashionMnistModel () 模型搭建



模型搭建与训练

- 模型搭建:

```
class FashionMnistModel(nn.Module):
    def __init__(self):
        '''
        *****请在此写入你的代码*****
        定义模型
        '''

    def forward(self, x):
        '''
        *****请在此处输入你的代码*****
        输入: input, 它的size是(batch_size, img_h, img_w, img_c)
        输出 (返回值): output(预测值), hidden(隐藏层的值)
            * output的size是(batch_size, num_label)

        定义模型函数:
        * 将输入经过卷积层和激活函数
        * 使用pooling降低通道数
        * 对卷积层的输出做适当的维度变换
        * 用线性层将output映射到num_label的维度上
        * 返回output
        '''
```




模型搭建与训练

- 模型训练:

```
def train(self, train_loader, test_loader):  
    '''  
    训练函数  
    '''  
    self.model.train()  
    for epoch in range(self.epochs):  
        loss_list = []  
        for batch_idx, (data, target) in enumerate(train_loader):  
            data, target = data.to(self.device), target.long().to(self.device)  
            self.optimizer.zero_grad()  
            output = self.model(data)  
            loss = self.loss_function(output, target)  
            loss.backward()  
            self.optimizer.step()  
            loss_list.append(loss.item())  
            if batch_idx % 50 == 0:  
                print('Train Epoch: {} [{}/{}] ({:.0f}%) \t Loss: {:.6f}'.format(  
                    epoch, batch_idx * len(data), len(train_loader.dataset),  
                    100. * batch_idx / len(train_loader), loss.item()))  
        self.test(test_loader)  
        # 保存模型参数  
        if epoch+1 % 5 == 0:  
            self._save_model(epoch+1)
```

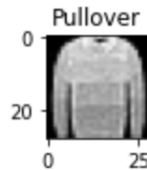
模型搭建与训练

- 模型测试:

```
def test(self, test_loader):  
    '''  
    检验模型测试集上的效果  
    '''  
    self.model.eval()  
    test_loss = 0  
    correct = 0  
    with torch.no_grad():  
        for data, target in test_loader:  
            data, target = data.to(self.device), target.long().to(self.device)  
            output = self.model(data)  
            test_loss += self.loss_function(output, target).item() # sum up batch loss  
            pred = output.argmax(dim=1, keepdim=True) # get the index of the max log-probability  
            correct += pred.eq(target.view_as(pred)).sum().item()  
    test_loss /= len(test_loader.dataset)  
    print('\nTest set: Average loss: {:.4f}, Accuracy: {}/{} ({:.0f}%) \n'.format(  
        test_loss, correct, len(test_loader.dataset),  
        100. * correct / len(test_loader.dataset)))
```

```
img, label = test_dataset.__getitem__(20)  
pred = inference(model2, img)  
fig = plt.figure(figsize=(1, 1))  
plt.imshow(np.squeeze(img), cmap='gray')  
plt.title(label_classes[pred])
```

Text(0.5, 1.0, 'Pullover')





回顾



人工智能与
Python程序设计

- 1. 卷积网络的训练流程
- 2. 简单卷积网络的搭建
- 3. 损失定义与模型优化
- 4. 图像识别大作业



谢谢！